

Lenguaje PL/SQL Oracle

PL/SQL es una de las tecnologías centrales de Oracle y es esencial para aprovechar todo el potencial de Oracle Database. PL/SQL combina las capacidades de acceso a datos relacionales del lenguaje de consulta estructurado (SQL) con un lenguaje de procedimiento incorporado flexible, que ejecuta consultas complejas y lógica programática dentro del propio motor de la base de datos. Esto mejora la agilidad, la eficiencia y el rendimiento de las aplicaciones basadas en bases de datos.

¿Qué es PL/SQL?

PL/SQL es un lenguaje de programación imperativo¹ y propietario de Oracle. PL/SQL es un lenguaje de programación ejecutado en los servidores Oracle con acceso a todas las sentencias SQL, incluye variables, funciones, control de flujo. Permite completar al lenguaje SQL. El propósito central del lenguaje Oracle PL/SQL es hacer que sea lo más fácil y eficiente posible consultar y cambiar el contenido de las tablas en una base de datos. Por supuesto, debe utilizar el lenguaje SQL para acceder a las tablas.

Cada sitio web sofisticado que se visita y cada aplicación que se ejecuta se construye a partir de una pila de tecnologías de software. En la cima de esta pila se encuentra la capa de presentación, las pantallas o dispositivos interactivos con los que el usuario interactúa directamente. (Un lenguaje popular para implementar capas de presentación es Java). En la base de la pila se encuentra el sistema operativo, es decir, el código máquina que se comunica con el hardware.

En algún punto intermedio de la pila tecnológica, se encuentra el software de base de datos que nos permite almacenar y manipular grandes volúmenes de datos complejos. La tecnología de bases de datos relacionales, basada en SQL, es la tecnología de bases de datos dominante en el mundo actual.

SQL, acrónimo de Lenguaje de Consulta Estructurado (Structured Query Language), es un lenguaje muy potente, orientado a conjuntos, cuyo único propósito es manipular el contenido de bases de datos relacionales. Si se escribe aplicaciones basadas en Base de Datos de Oracle, se debe ejecutar sentencias SQL para recuperar o modificar datos de esa base de datos. Sin embargo, SQL no puede utilizarse para implementar toda la lógica de negocio y funcionalidades necesarias de las aplicaciones.

PL/SQL significa Lenguaje Procedimental/Lenguaje de Consulta Estructurado (*Procedural Language/Structured Query Language*). PL/SQL ofrece un conjunto de comandos procedimentales (sentencias IF, bucles, asignaciones), organizados en bloques, que complementan y amplían el alcance de SQL. PL/SQL es la extensión de SQL de Oracle, diseñada para desarrolladores que trabajan con Base de Datos de Oracle.

Es posible crear aplicaciones sobre SQL y Base de Datos de Oracle sin usar PL/SQL. Sin embargo, usar PL/SQL para realizar operaciones específicas de la base de datos, en particular la ejecución de sentencias SQL, ofrece varias ventajas, como una estrecha integración con SQL, un mejor rendimiento gracias a la reducción del tráfico de red y la portabilidad. (Los programas PL/SQL pueden ejecutarse en cualquier instancia de Base de Datos de Oracle). Por lo tanto, el código FRONTEND de muchas aplicaciones ejecuta tanto las sentencias SQL como bloques PL/SQL para maximizar el rendimiento y mejorar la mantenibilidad de dichas aplicaciones.

Componentes básicos de los programas PL/SQL - Construyendo con bloques en PL/SQL

PL/SQL es un lenguaje estructurado por bloques. Un bloque PL/SQL se define mediante las palabras clave DECLARE, BEGIN, EXCEPTION y END, que dividen el bloque en tres secciones:

¹ Un **lenguaje imperativo** es un tipo de lenguaje de programación. En este tipo de lenguajes, las instrucciones se ejecutan unas tras otras, de manera secuencial, salvo cuando encuentra estructuras de control condicionales o bucles. Hay declaración de variables, tipos y procedimientos, aunque esto varía notablemente en función del lenguaje utilizado, pues hay los que exigen las declaraciones mientras que otros permiten que esos elementos no sean declarados.

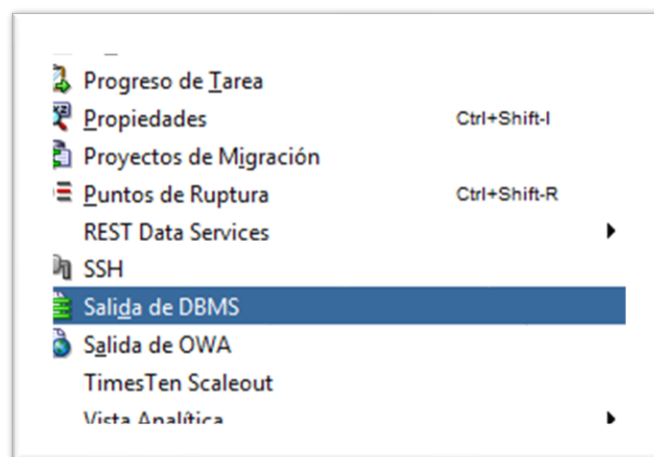
1. **Declarativo:** Sentencias que declaran variables, constantes y otros elementos de código, que pueden usarse dentro de ese bloque.
2. **Ejecutable:** Sentencias que se ejecutan al ejecutarse el bloque. Única sección obligatoria.
3. **Manejo de excepciones:** Una sección especialmente estructurada que permite capturar cualquier excepción generada al ejecutarse la sección ejecutable (errores de lógica de aplicación, fallas de hardware o de red, errores de integridad de datos).

Solo se requiere la sección ejecutable. No es necesario declarar nada en un bloque ni capturar las excepciones generadas en él.

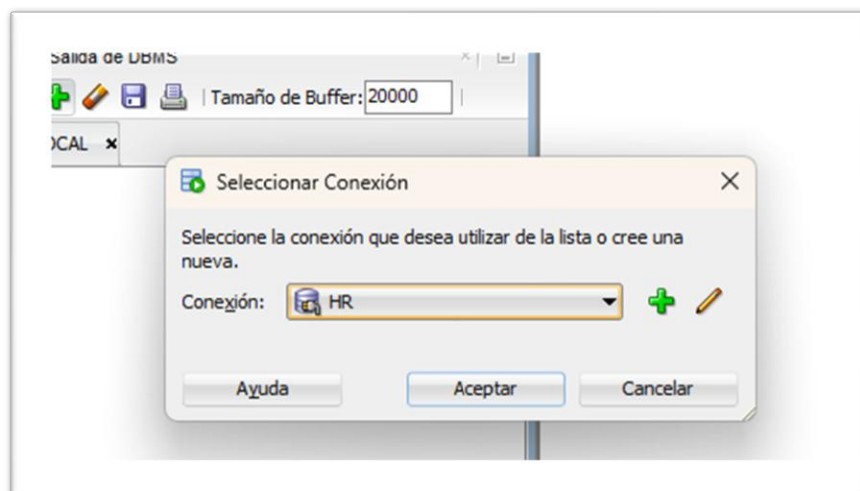
Un bloque en sí mismo es una sentencia ejecutable, una unidad, por lo que se pueden anidar bloques dentro de otros bloques. Los bloques pueden almacenarse o no en la Base de Datos, a estos últimos se los conoce como bloques anónimos por no tener nombre.

Vemos algunos ejemplos:

Antes de comenzar, aclaremos que PL/SQL no contiene instrucciones de Entrada-Salida, por lo que se utiliza un paquete estándar que trae las Bases de Datos Oracle, el paquete² DBMS_OUTPUT. El paquete DBMS_OUTPUT permite enviar mensajes por consola desde los bloques anónimos o almacenados y los paquetes. Es especialmente útil para mostrar información de depuración de PL/SQL y con el procedimiento de salida PUT_LINE. Para poder visualizar la consola DBMS en el SQL Developer se debe activar desde el menú VER, Salida de DBMS:



Se abrirá una ventana donde se debe elegir la conexión al esquema que se desea visualizar:



² Un paquete es un contenedor para múltiples funciones y procedimientos. Oracle extiende PL/SQL con numerosos paquetes suministrados o integrados.

Este procedimiento estándar se utiliza colocando entre paréntesis lo que se desea visualizar, sea esto constantes, literales, contenidos de variables o una concatenación de estos elementos. Usemos este paquete en nuestro primer bloque anónimo:

```
BEGIN
    DBMS_OUTPUT.put_line ('Hola a Todos!');
END;
```

El siguiente ejemplo de bloque anónimo declara una variable de tipo VARCHAR2 (cadena) con una longitud máxima de 100 bytes para contener la cadena 'Hola a Todos!'. DBMS_OUTPUT.PUT_LINE acepta la variable, en lugar de la cadena literal, para su visualización:

```
DECLARE
    l_mensaje VARCHAR2 (100) := 'Hola a Todos!';
BEGIN
    DBMS_OUTPUT.put_line (l_mensaje);
END;
```

Tenga en cuenta que se ha nombrado la variable l_mensaje. Generalmente uso el prefijo l_ para variables locales (variables definidas dentro de un bloque de código) y el prefijo g_ para variables globales definidas en un paquete. Pero estas convenciones se establecen en cada grupo de trabajo.

El siguiente bloque de ejemplo agrega una sección de excepción que captura cualquier excepción (errores de cualquier tipo, usando WHEN OTHERS) que pueda generarse y muestra el mensaje de error, que devuelve la función SQLERRM (proporcionada por Oracle):

```
DECLARE
    l_mensaje VARCHAR2 (100) := 'Hola a Todos!';
BEGIN
    DBMS_OUTPUT.put_line (l_mensaje);
EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.put_line (SQLERRM);
END;
```

El siguiente bloque de ejemplo demuestra la capacidad de PL/SQL para anidar bloques dentro de bloques, así como el uso del operador de concatenación (||) para unir varias cadenas:

```
DECLARE
    l_mensaje VARCHAR2 (100) := 'Hola ';
BEGIN
    DECLARE
        l_mensaje2 VARCHAR2 (100) := l_mensaje || 'a Todos!';
    BEGIN
        DBMS_OUTPUT.put_line (l_message2);
    END;
EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.put_line (DBMS_UTILITY.format_error_stack);
END;
```

Ejecución de bloques PL/SQL

Una vez escrito un bloque de código PL/SQL, puede ejecutarlo. Existen muchas herramientas diferentes para ejecutar código PL/SQL si no está utilizando el SQL Developer. La más básica es SQL*Plus, una interfaz de línea de comandos para ejecutar sentencias SQL y bloques PL/SQL. Esta consola de comandos viene por defecto cuando se instala la Base de Datos Express 21c Oracle.

Lo primero que hay que hacer tras conectarse a la base de datos mediante SQL*Plus es activar la salida del servidor, de modo que las llamadas a DBMS_OUTPUT.PUT_LINE muestren texto en pantalla. A continuación, escribo el código que constituye mi bloque, línea por línea ingresándola con Enter. Finalmente, introduzco una barra diagonal (/) para indicar a SQL*Plus que ejecute este bloque.

SQL*Plus ejecuta el bloque y muestra la salida en la pantalla. Oracle proporciona SQL*Plus como una especie de entorno de mínimo común denominador para ejecutar sentencias SQL y bloques PL/SQL. Algunos desarrolladores siguen utilizando únicamente SQL*Plus, pero la mayoría utiliza un entorno de desarrollo integrado (IDE).

Cada herramienta ofrece ventanas y pasos ligeramente diferentes para crear, guardar y ejecutar bloques PL/SQL, así como para activar y desactivar la salida del servidor. Se tendrá en cuenta que en este curso se utiliza SQL Developer como aplicación Cliente con la Base de Datos y como entorno de desarrollo.

Nota: Aun cuando no se utilice SQL*PLUS para conectarse con la Base de Datos debe terminar todos los bloques anónimos con “/”.

Acerca de los nombres en Oracle Database

Ahora que se comprende la importancia de asignar nombres a la lógica, debemos hablar sobre las reglas para nombres (identificadores) tanto en PL/SQL como, en general, en Oracle Database.

A continuación, se presentan las reglas para construir identificadores válidos en Oracle Database:

- La longitud máxima es de 30 caracteres.
- El primer carácter debe ser una letra, pero cada carácter posterior puede ser una letra, un número (del 0 al 9), un signo de dólar (\$), un guion bajo (_) o un signo de número (#). Todos los siguientes son identificadores válidos: saludos_alumnos, saludos#alumnos, saludos#alumnos. Sin embargo, estos no son válidos: 1saludos_alumnos, saludos%alumnos.
- PL/SQL no distingue entre mayúsculas y minúsculas en lo que respecta a los identificadores. PL/SQL trata todos los siguientes como el mismo identificador: saludos_alumnos, Saludos_Alumnos, SALUDOS_ALUMNOS.

Para ofrecerle mayor flexibilidad, Oracle Database permite eludir las restricciones de la segunda y la tercera regla al encerrar su identificador entre comillas dobles. Un identificador entre comillas puede contener cualquier secuencia de caracteres imprimibles, excluyendo las comillas dobles; también se conservarán las diferencias entre mayúsculas y minúsculas. Por lo tanto, todas las siguientes cadenas son identificadores válidos y distintos: "Abc", "ABC", "a b c"

Rara vez encontraremos identificadores entre comillas en código PL/SQL. Algunos grupos de desarrollo los utilizan para ajustarse a sus convenciones de nomenclatura o porque consideran que las cadenas con mayúsculas y minúsculas son más fáciles de leer.

Estas mismas reglas se aplican a los nombres de los objetos de la Base de Datos, como Tablas, Vistas y Procedimientos, con una regla adicional: a menos que se escriban entre comillas dobles los nombres de dichos objetos, Oracle Database los almacenará en mayúsculas. Por lo tanto, cuando se creó el procedimiento **saludos** se creó como **SALUDOS**. Pero en las siguientes llamadas a dicho procedimiento tomará en las tres el procedimiento **SALUDOS**:

```
BEGIN
  saludos;
  Saludos;
  SALUDOS;
  "SALUDOS";
END;
```

Sin embargo, Oracle no podrá llamar al procedimiento **SALUDOS** si lo invocamos de la siguiente forma:

```
BEGIN  
    "saludos";  
END;
```

Ya que buscará en la Base de Datos un procedimiento llamado así **saludos** y no existe. Si se desea que el nombre tenga un formato específico, se deberá crear el procedimiento con el nombre entre comillas dobles:

```
CREATE OR REPLACE PROCEDURE "Hello_World" ...
```

Hay un aspecto en el que PL/SQL se parece a cualquier otro lenguaje de programación: el motor de ejecución de PL/SQL sólo hace exactamente lo que se le indica. Cada bloque de código PL/SQL que se escribe contiene una o más sentencias que implementan reglas o procesos de negocio complejos. Al ejecutar un bloque de código, ya sea como un bloque anónimo, un script o llamando a una unidad de programa almacenada que contiene toda la lógica, Oracle Database sigue las instrucciones que se especifican en ese bloque.

Por lo tanto, es fundamental saber cómo especificar las sentencias qué deben ejecutarse, en qué circunstancias y con qué frecuencia. Para ello, Oracle Database ofrece construcciones condicionales e iterativas.

Bifurcación Condicional en el Código

Casi todo el código que se escribe requiere control condicional, es decir, la capacidad de dirigir el flujo de ejecución del programa según una condición. Esto se consigue con las sentencias IF-THEN-ELSE y CASE.

También existen expresiones CASE; aunque no son lo mismo que las sentencias CASE, a veces se pueden usar para eliminar por completo la necesidad de una sentencia IF o CASE.

IF

La sentencia IF permite implementar lógica de ramificación condicional en los programas. Con ella, se podrán implementar requisitos como los siguientes:

- Si el salario está entre \$10,000 y \$20,000, aplicar una bonificación de \$1,500.
- Si la colección contiene más de 100 elementos, truncarla.

La sentencia IF se presenta en tres variantes:

Tipo de IF	Características
IF <i>cond</i> THEN END IF;	Esta es la forma más simple de la sentencia IF. La condición entre IF y THEN determina si el conjunto de sentencias entre THEN y END IF debe ejecutarse. Si la condición se evalúa como FALSE o NULL, el código no se ejecutará.
IF <i>cond</i> THEN ELSE END IF;	Esta combinación implementa la lógica de "o esto o lo otro": según la condición entre las palabras clave IF y THEN, se ejecuta el código entre THEN y ELSE o entre ELSE y END IF. Se ejecuta una de estas dos secciones de sentencias.
IF <i>cond</i> THEN ELSIF <i>cond</i> THEN ELSE END IF;	Esta última y más compleja forma de la sentencia IF selecciona una condición verdadera de una serie de condiciones mutuamente excluyentes y luego ejecuta el conjunto de sentencias asociadas a dicha condición. Si escribe sentencias IF como esta debería considerar usar sentencias CASE con búsqueda.

IF-THEN.

En la sentencia IF_THEN se evalúa la condición y si resulta **verdadera**, se ejecutan uno o más líneas de código de programa, las que figuren entre **THEN** y **END IF;**. En el caso de que la condición resulte **falsa o nula**, no se realiza NINGUNA acción. Luego de haber evaluado la condición y de haber ejecutado o

no la/las sentencias de la bifurcación el programa seguirá ejecutando la siguiente sentencia del END IF; La siguiente sentencia compara dos valores numéricos. Tenga en cuenta que si uno de estos dos valores es NULL, la expresión completa devolverá NULL. En el siguiente ejemplo, la bonificación no se otorga cuando el salario es nulo:

```
IF I_salario > 40000 THEN  
    v_bono:=500;  
END IF;
```

Existen excepciones a la regla de que un valor nulo en una expresión booleana da como resultado un valor nulo. Algunos operadores y funciones están diseñados específicamente para gestionar valores nulos de forma que generen resultados verdaderos y falsos (y no nulos). Por ejemplo, puede usarse "IS NULL" para comprobar la presencia de un valor NULL haciendo que el resultado sea Verdadero o Falso y no Null:

```
IF I_salario > 40000 OR I_salario IS NULL THEN  
    v_bono:=500;  
END IF;
```

En este ejemplo, "salario IS NULL" se evalúa como VERDADERO si el salario no tiene valor; de lo contrario, se evalúa como FALSO. Los empleados cuyos salarios no se indiquen ahora también recibirán bonificaciones.

IF-THEN-ELSE

Se evalúa la condición y si resulta **verdadera**, se ejecutan una o más líneas de código de programa. En el caso de que la condición resulte **falsa**, se ejecutan las instrucciones que siguen a la instrucción ELSE. Sólo se permite una instrucción ELSE en cada instrucción IF. Veamos un ejemplo de la construcción IF-THEN-ELSE:

```
IF I_salario <= 40000 THEN  
    v_bono:= 0;  
ELSE  
    v_bono:= 500;  
END IF;
```

En este ejemplo, los empleados con un salario superior a \$40,000 recibirán una bonificación de \$500, mientras que los demás no recibirán ninguna. ¿Qué ocurre si, por cualquier motivo, el salario de un empleado resulta ser nulo? En ese caso, se ejecutarán las instrucciones posteriores a ELSE y el empleado en cuestión recibirá la bonificación, que se supone sólo se otorga a los empleados con salarios altos.

Si el salario pudiera ser nulo, puede protegerse contra este problema utilizando la función NVL. La función **NVL()** de **Oracle**, es una herramienta fundamental para la manipulación y transformación de datos en **SQL** y **PLSQL**, permite reemplazar los valores nulos con una alternativa más significativa, lo que resulta especialmente útil en consultas y procesos de datos donde es necesario trabajar con valores no nulos.

La función **NVL()** de **Oracle** acepta dos argumentos y devuelve el valor del primer argumento si no es nulo. Si el primer argumento es nulo, devuelve el segundo argumento como valor de reemplazo. Esta flexibilidad nos permite realizar operaciones y cálculos precisos, asegurando que los resultados sean coherentes y fiables, incluso en la presencia de valores nulos.

```
IF NVL(I_salario,0) <= 40000 THEN  
    V_bono:=0;  
ELSE  
    v_bono:= 500;  
END IF;
```

La función NVL devolverá 0 siempre que el salario sea nulo, lo que garantiza que los empleados con un salario nulo tampoco reciban bonificación (esos empleados de bajos recursos).

Es importante recordar que una de las dos secuencias de instrucciones siempre se ejecutará, ya que IF-THEN-ELSE es una construcción de tipo "o esto o aquello". Una vez ejecutado el conjunto apropiado de sentencias, el control pasa a la sentencia inmediatamente posterior a END IF.

IF-ELSIF

Veamos ahora el uso de ELSIF. Esta última forma de la sentencia IF resulta útil cuando se necesita implementar lógica con múltiples alternativas; no se trata de una situación excluyente. La formulación IF-ELSIF permite gestionar múltiples condiciones dentro de una sola sentencia IF. En general, se recomienda usar ELSIF con alternativas mutuamente excluyentes (es decir, solo una condición puede ser VERDADERA para cualquier ejecución de la sentencia IF). El mismo resultado podríamos obtener si realizamos IFs anidados.

Cada cláusula ELSIF debe tener un THEN después de su condición. La palabra clave ELSE no la necesita. La cláusula ELSE en IF-ELSIF es la cláusula "de lo contrario" de la sentencia. Si ninguna de las condiciones se evalúa como VERDADERA, se ejecutarán las sentencias de la cláusula ELSE. Sin embargo, la cláusula ELSE es opcional. Se puede codificar un IF-ELSIF que solo contenga las cláusulas IF y ELSIF. En tal caso, si ninguna de las condiciones es VERDADERA, no se ejecutará ninguna instrucción dentro del bloque IF.

A continuación, se muestra un ejemplo de una instrucción IF-ELSIF que verifica tres condiciones diferentes y contiene una cláusula ELSE en caso de que ninguna de ellas sea VERDADERA.

```
IF I_salario BETWEEN 10000 AND 20000 THEN
    v_bono:= 1000;
ELSIF I_salario > 20000 THEN
    v_bono:= 500;
ELSE
    v_bono:= 0;
END IF;
```

Ejercicios de Bloques Anónimos

1. Desarrollar un bloque que dado un valor numérico indique si el mismo es positivo.
2. Desarrollar un bloque que dado un valor numérico indique si el mismo es positivo, negativo o cero.
3. Escribir un programa que declare un número entero y muestre por pantalla si es par o impar.
4. Escribir un bloque que declare un número entero y muestre por pantalla si es múltiplo de 3 o no.
5. Escribir un bloque que declare un número entero y muestre por pantalla si es múltiplo de 3 caso contrario mostrar el número múltiplo de tres anterior al dado.
6. Realizar un bloque anónimo que declare dos variables numéricas cuyo valor inicial sea cero y se le asigne su valor en el procedimiento y muestre el resultado de la suma.
7. Realizar un bloque anónimo que declare dos variables numéricas con sus valores iniciales en la declaración. En la sección ejecutables se deberán comparar las variables indicando si la primera es mayor, menor o igual a la segunda variable mediante una leyenda adecuada.
8. Crear un bloque PL que de un nombre y un apellido devuelva las iniciales de la persona.
9. Escribir un bloque con dos variables numéricas que muestre la división del mayor con el menor de los valores. Si el divisor es cero el programa debe mostrar un error.
10. Para tributar un determinado impuesto se debe ser mayor de 18 años y tener unos ingresos iguales o superiores a \$ 800.000 mensuales. Escribir un bloque que controle edad e ingresos mensuales y muestre por pantalla si se tiene que tributar impuestos o no.